`yarg-sync` — the sync client

`yarg-sync` pulls a `yarg-song-server` library into an ordinary local songs folder. **YARG is not modified and does not know the server exists.** It sees files in a folder, which is all it has ever needed to see.

That is the whole point of Phase 2b: shared libraries work today, on stock YARG, on every platform YARG runs on, with no client fork and no risk to anyone's install.

```
yarg-sync -server http://pi.local:8080 -songs "%USERPROFILE%\YARG Songs"
```

Flags

Flag	Default	Meaning
`-server`	*(required)*	Base URL of a `yarg-song-server`.
`-songs`	`./songs`	Local folder to sync into.
`-dry-run`	off	Report what would change; write nothing.
`-prune`	**off**	Delete songs the server no longer has.
`-version`		Print version and exit.

Exit status is `0` only if every song the server offered was fetched and verified. A run that fetched some songs and failed on others exits `1`. A sync that silently skipped half a library is exactly the kind of green this project audits other people's scripts for.

What it will and will not touch

The client writes exactly one shape of file:

```
<chart_hash>.sng          # 40 lowercase hex characters, then ".sng"
```

Anything else in that folder is not ours and is never read, renamed, rewritten or deleted — not by a normal run, and not by `-prune`. That includes a loose song folder the player charted themselves, a `.sng` they named by hand, and a file whose name is nearly ours but not quite. `internal/e2e` (`TestThePlayersOwnSongsAreNeverTouched`) snapshots such a folder, syncs, prunes, and compares every byte.

So it is safe to point `-songs` at a folder the player already uses. That was a design requirement, not a happy accident: a sync tool that can eat somebody's own charts is one nobody should run.

`-prune` is off by default and costs an extra request (a second `POST /api/v1/have` with an empty list, to learn the server's full set). Deleting a player's music because the server was briefly reachable but empty is a failure mode worth paying a round trip to avoid.

Why files are named by chart hash

Song identity in this project is `SHA1(chart file bytes)` and nothing else — the same rule the server indexes by and the same rule YARG uses. Naming the file after it makes the local folder self-describing: the client can take inventory by reading directory entries, with no state file to corrupt, lose or disagree with reality.

It also means the client can check the server's work. Every download lands as `<hash>.sng.part`, is opened and scanned locally, and is only renamed into place if the identity re-derived from the bytes matches the one that was asked for. A truncated transfer, a proxy serving something else, or a server bug all fail closed. `TestCorruptDownloadIsRefused` and `TestWrongSongIsRefused` cover it.

When two packages share a chart

Two folders can hold the same chart with different extras — different album art, a different `charter`, an extra stem. Identity is the chart, so the client wants one copy, and the server refuses to guess: `GET /song/{hash}.sng` answers **300 Multiple Choices** with the candidates. The client then picks the lowest package hash, which is arbitrary but *deterministic*: two machines syncing the same server pick the same package.

Picking the same package is half of a shared library. The other half is that the server encodes that package the same way every time, and until 2026-09-06 it did not — `PackDir` drew a random obfuscation mask per call, so two machines that had agreed perfectly about *which* song still received different files. It was found by running exactly this client on two computers and comparing SHA-256s: 16 of 22 differed. The mask is now derived from the package hash. See `docs/TEST-CORPUS.md`, fourth oracle run.

Windows Defender flagged the binary once, then stopped (false positive)

On 2026-09-05 between 20:26 and 20:27, four Defender detections fired on ENG-1 against `cmd/yarg-sync` builds:

```
Trojan:Win32/Bearfoos.A!ml      (ThreatID 2147731250)
```

`go build ./cmd/yarg-sync` failed with "*the file contains a virus or potentially unwanted software*" before it could link, twice; the finished binary was quarantined twice more.

By 21:10 the same day it no longer reproduced, with nothing changed on the machine. Measured rather than assumed:

Test	Result
3 × `go clean -cache` then build to `bin\`, then execute	built, survived, ran
3 × build with a different `-ldflags` version string (3 distinct SHA-256s)	built, survived, ran
New detections across all six	**0**
Defender signature version before and after	identical (`1.459.64.0`)
Exclusions added	**none**

Three distinct hashes ruling it out means this was not a per-file verdict cached clean — the classifier's answer itself changed. `!ml` verdicts are cloud-delivered and get revised upstream; this one was, within the hour.

What to do if it comes back

Re-measure first. The detection above would have justified a permanent Defender exclusion if anyone had acted on it immediately, and that exclusion would still be there now, weakening the machine for a problem that had already evaporated. An `!ml` verdict is provisional by construction.

If it recurs *and persists across a re-test*:

- **Do not add a Defender exclusion**, do not allow-list the threat ID (that allows the whole `Bearfoos.A!ml` family machine-wide, including true positives), and never suggest disabling protection to get a build through.
- **Submit the binary to Microsoft's false-positive form.** Free, and it fixes the verdict for every player who downloads a release, not just for us.
- **Sign the release artifact.** A code-signing certificate is the actual solution for an unsigned executable that strangers download. Note that a self-signed certificate does *not* help here — it does nothing for an ML verdict.

The signing identity is decided: FatalException. This is a personal project and is signed as one; it is never signed as Juniper Design Group, whose certificate would attach a company's name and liability to a personal release. What is still open is which *kind* of certificate — the options differ mainly in how quickly SmartScreen reputation accrues, and some require a registered legal entity rather than an individual, which is the question to settle before pricing anything. - It is not a CI problem in any case. The runner is Linux; `release` and the container image are unaffected.

`internal/e2e` still exercises the client in-process rather than executing the built binary. That was originally a workaround for this detection; it is worth keeping on its own merits, since it makes the suite independent of whatever any scanner thinks of a freshly linked executable on any given day.

How this is verified

`internal/syncclient` unit-tests the client against HTTP stubs. Stubs prove the client's own logic, but a stub only ever says what its author believed the server says — it cannot catch the two sides disagreeing about the wire.

`internal/e2e` therefore wires the real `httpapi.Server`, the real `library.Build` and the real `packcache` to the real client and runs whole syncs against a real library on disk:

Test	What would have to break for it to fail
`TestWholeSyncRoundTrip`	Client ends up with every song, each archive readable and each one the song its name claims — identity re-derived from the downloaded bytes.
`TestSecondRunDownloadsNothing`	Sync is idempotent; an up-to-date client transfers zero song bytes.
`TestThePlayersOwnSongsAreNeverTouched`	Nothing unmanaged is altered, including across a `-prune`.
`TestDryRunWritesNothing`	`-dry-run` names every song and writes no entries.
`TestSharedChartHashSyncsOnce`	The 300 Multiple Choices exchange completes and yields one archive.

Each was red-proofed on 2026-09-05 by breaking the code it covers and confirming that test, and only that test, failed:

- `managedName` loosened from `^[0-9a-f]{40}\.sng$` to `\.sng$` → `TestThePlayersOwnSongsAreNeverTouched` alone failed.
- the `-dry-run` guard replaced with `if false` → `TestDryRunWritesNothing` alone failed.

A test that has never been seen to fail is a test whose meaning is unmeasured.